Software Architecture and Design Specification

Project: Chat Application (Socket Programming)

Version: 1.0

Authors: Team 8 – Annanya Sharma, Aryan Shashank Musale, Kaushik Krishna Dwarakanath (Lead), Kachhadiya Darshil Vipulkumar

Date: 2 October 2026

Status: Draft / For Design Review

Revision History

Version	Date	Author	Change Summary
1.0	02-10-2026	Team 8	Architecture and design specification for Chat Application

Approvals

Role	Name	Signature/Date
Team Lead	Kaushik Krishna Dwarakanath	
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Chat Application (Socket Programming). It translates the approved SRS requirements into a modular system architecture, component interfaces, UML models, security architecture, and detailed design.

1.2 Scope

Covers user registration and authentication, real-time messaging, typing indicators, message history, profile management, REST interfaces, WebSocket communication, and MongoDB persistence.

1.3 Audience
Development team, testers/QA, security reviewers, course evaluators, and team members responsible for implementation and maintenance.

1.4 Definitions
- Socket — A persistent, bidirectional connection between client and server used for real-time data exchange.
- Room — A logical chat channel that groups a set of connected users who can message each other.
- Presence — The real-time online/offline status of a user.
- JWT — JSON Web Token, a signed token used to authenticate a user across REST and socket requests.
- Typing indicator — A transient UI signal shown to other users when someone is composing a message.
- Message history — The persisted, ordered log of past messages in a room, retrievable on room join.

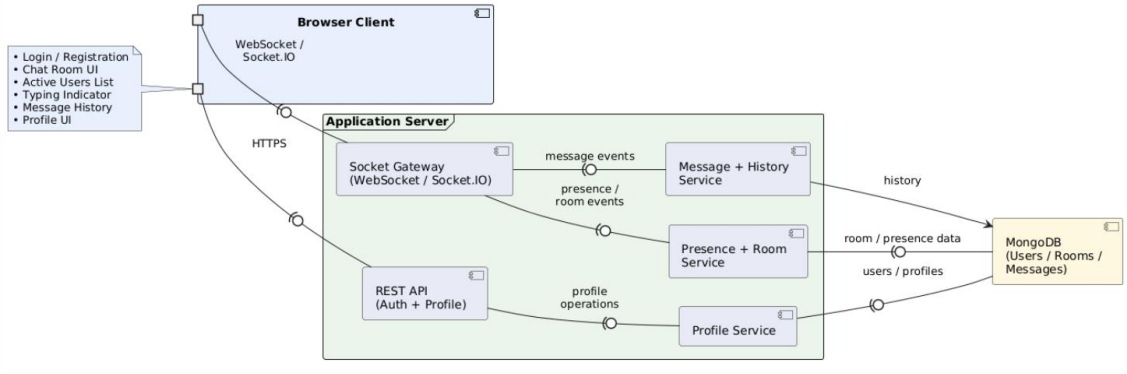
2. Document Overview
2.1 How to use this document
Use this document as the design reference for implementation, integration, testing, and review. It provides the architecture, component responsibilities, interfaces, UML diagrams, security considerations, and traceability from SRS requirements to design elements.

2.2 Related Documents
Team 8 Software Requirements Specification (SRS) v1.0; Software Test Plan (STP) v1.0; Requirements Traceability Matrix (RTM) in the SRS; Deliverable 1 – Synopsis / Project Proposal.

3. Architecture
3.1 Goals & Constraints
Goals: real-time message delivery, modularity, maintainability, secure authentication, reliable reconnection. Constraints: socket-based (non-polling) communication, 50-concurrent-user capacity target, single-host demo deployment, browser-only client, dependence on third-party libraries (Socket.IO, Express/Flask, MongoDB driver)

3.2 Stakeholders & Concerns
Users: simple and responsive chat experience.
Development team: modularity, clear interfaces, and parallel development.
Testing/CI responsibility: testability, traceability, and repeatable validation.
Security/API review: authentication, authorization, input validation, and safe rendering.
Database responsibility: reliable message and user persistence.

3.3 Component (UML) Diagram



The architecture uses browser clients connected to a central application server. REST is used for authentication/profile operations, while WebSocket/Socket.IO provides real-time event communication. Application services use MongoDB for persistent users, rooms, and messages.

3.4 Component Descriptions

- Browser Client: provides login/registration, chat room, active user list, typing indicator, message history, and profile UI.
- REST API: handles registration, login, session-token support, and profile updates over HTTPS.
- Socket Gateway: manages authenticated WebSocket/Socket.IO connections and real-time events such as connect, disconnect, sendMessage, typing, and presenceUpdate.
- Presence + Room Service: tracks online/offline state and manages membership in the shared chat room.
- Message + History Service: validates messages, persists message records, and retrieves recent message history.
- Profile Service: validates and persists basic profile updates.
- MongoDB: stores users, rooms, messages, and related timestamps.

3.5 Chosen Architecture Pattern and Rationale

Client–Server architecture is the primary architecture pattern because browser clients communicate with a central server that manages authentication, socket connections, shared-room state, and database access. The real-time part uses event-driven communication through Socket.IO/WebSockets, which is suitable for asynchronous and reactive communication. The server is also organized into modules/layers so UI/API handling, application services, and persistence remain separated. Microservices is not

selected because the current project is a small, single-room academic application and would add unnecessary operational complexity.

3.6 Technology Stack & Data Stores

Layer / Component	Selected Technology / Design
Frontend	HTML5, CSS3, JavaScript, Socket.IO Client; modern web browser
Backend	Node.js + Express + Socket.IO (primary design); Python + Flask-SocketIO is an alternative in the project proposal
Communication	HTTPS for REST; WebSocket via Socket.IO for real-time events
Database	MongoDB; Mongoose for Node.js or PyMongo for Flask
Version Control / Project Tools	Git + GitHub; Jira/GitHub Projects as planned course tooling
Testing / Quality	Jest or Pytest; GitHub Actions; SonarCloud as course static-analysis tooling

3.7 Risks & Mitigations

Risk	Mitigation
Socket/network interruption	Use reconnect handling and verify NFR-004 through integration/failure tests.
Invalid or expired authentication token	Validate the session token before socket access and reject invalid/expired tokens.
Unsafe message content	Validate and sanitize user-supplied content before broadcast/storage and escape XSS payloads before rendering.
Database/history failure	Keep database access isolated behind the history/data layer and test message persistence and retrieval.

3.8 Traceability to Requirements

Design ID	Design Element	SRS Requirement Group
DS-Auth-01	REST Auth API (registration, login, password hashing)	FR-001, FR-002, FR-003
DS-Auth-02	JWT issue and validation	FR-004
DS-Sock-01	Socket Gateway (connection handling)	FR-005, FR-007

DS-Msg-01	Message Service (broadcast)	FR-006
DS-Pres-01	Presence + Room Service (presence, active-user list)	FR-008, FR-014
DS-Type-01	Typing handler + Chat UI indicator	FR-009, FR-012
DS-Hist-01	Message + History Service, MongoDB	FR-010, FR-013
DS-Room-01	Room Service (join shared room)	FR-011
DS-Prof-01	Profile Service + REST API	FR-015
DS-Perf-01	Socket Gateway, load configuration	NFR-001
DS-Avail-01	Application server deployment	NFR-002
DS-UX-01	Browser Client UI flow	NFR-003
DS-Rel-01	Reconnect handling (Gateway + Client)	NFR-004
DS-Maint-01	Module separation and unit-test structure	NFR-005
DS-Sec-01	Password hashing in the REST Auth API (only the hash is stored)	SEC-001
DS-Sec-02	Socket Gateway handshake check: JWT required before connecting or joining the room	SEC-002
DS-Sec-03	Message validation and sanitization in the Message + History Service	SEC-003
DS-Sec-04	JWT expiry/validity check on REST and socket endpoints (rejects invalid or expired tokens)	SEC-004
DS-Sec-05	Output escaping of message content in the Browser Client before rendering	SEC-005
DS-Sec-06	Security logging of failed logins and unauthorized socket attempts	SEC-006

3.9 Security Architecture

Security architecture is based on the security requirements in the SRS and the course security-validation approach. STRIDE is used as the threat-modeling structure.

Threat	**Relevant Design Control / Validation**
Spoofing	JWT/session-token authentication for REST and socket access; invalid/expired tokens are rejected.
Tampering	User-supplied message content is validated and sanitized before broadcast and storage.
Repudiation	Failed login and unauthorized socket connection attempts are logged.
Information Disclosure	REST authentication uses HTTPS; passwords are stored as hashes rather than plaintext.
Denial of Service	Per-user/IP rate limiting on login and `sendMessage` events; maximum message length and payload size enforced before processing;

Elevation of Privilege	Only authenticated users may join the room and access chat functions.

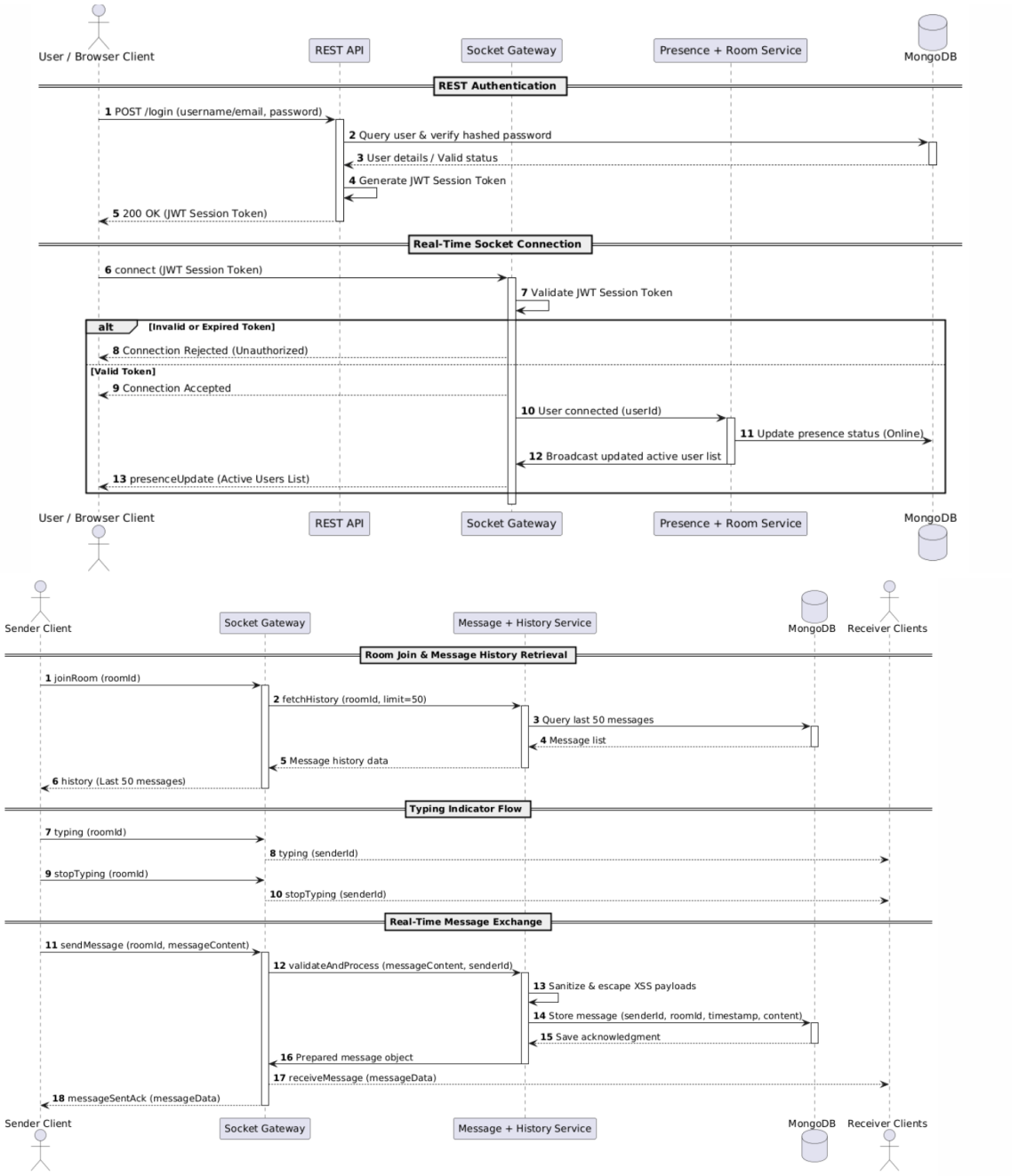
4. Design

4.1 Design Overview

The design decomposes the application into authentication, socket gateway, room/presence, messaging/history, profile, and persistence modules. Interfaces are kept explicit so the four team members can work in parallel while maintaining modularity, cohesion, and controlled coupling.

4.2 UML Sequence Diagrams

The following two sequence diagrams cover the major authentication/connection flow and the core real-time messaging flow, satisfying the prescribed template requirement for at least two project-specific sequence diagrams.



4.3 API Design

The SRS defines two main software interfaces: a REST API for authentication/profile operations and a Socket.IO API for real-time events. The following interfaces are the design contracts to be implemented.

Endpoint	Method	Request	Response	Error
/register	POST	{username, email, password}	201 {userId, username, email}	400 invalid or missing fields; 409 username/email already exists
/login	POST	{identifier, password} (identifier = email or username)	200 {token, user:{userId, username}}	400 missing fields; 401 invalid credentials
/profile	PUT	{displayName?, bio?, avatar?} + Bearer token	200 {userId, username, displayName, bio, avatar}	400 invalid fields; 401 missing/invalid/expired token
/messages?limit=50	GET	Bearer token	200 [{messageId, senderId, username, roomId, content, timestamp}], oldest to newest	401 missing/invalid/expired token

4.4 Error Handling, Logging & Monitoring

- Return clear authentication and validation errors without exposing sensitive information.
- Reject invalid or expired session tokens on REST and socket endpoints.
- Reject empty/invalid message content before broadcast or storage.
- Escape unsafe message content before client-side rendering.
- Log failed login and unauthorized socket connection attempts.
- Monitor message delivery time, concurrent-user performance, availability during the demo window, reconnect behavior, and message-history retrieval.

4.5 UX Design

The UI is organized around two primary screens: Login/Registration and the shared Chat Room. The chat room contains the message list, message input, active-user list, and typing-

indicator strip. The design aims for quick communication with minimal navigation, consistent feedback for validation errors, and clear online/offline state.

4.6 Open Issues & Next Steps

- Confirm the final backend choice: Node.js + Express + Socket.IO or Python + Flask-SocketIO.
- Finalize the exact MongoDB schema and indexes for users, rooms, and messages.
- Complete implementation-level API details and error codes during design/implementation.
- Update the SRS RTM with final design module names and test evidence as implementation progresses.
- Validate the design through component/integration tests, security validation, performance testing, and static analysis.

5. Appendices

5.1 Glossary

Term	Definition
Socket	Persistent bidirectional connection used for real-time data exchange.
Socket.IO	Library/protocol layer used for real-time client-server events.
JWT	Signed session token used to authenticate a user across REST and socket requests.
Presence	Real-time online/offline state associated with active socket connections.
REST API	HTTP-based interface used for registration, login, and profile operations.
RTM	Requirements Traceability Matrix mapping requirements to design and tests.

5.2 References

- Team 8 Software Requirements Specification (SRS) v1.0.
- Team 8 Deliverable 1 – Synopsis / Project Proposal.
- PES University Software Engineering course slides – Unit 1 and Unit 2.
- Software Engineering course SAD template.

5.3 Tools

- draw.io / PlantUML-style UML modelling for diagrams.
- Git + GitHub for version control.
- Jira / GitHub Projects for Agile planning.
- Postman for API testing.
- Jest / Pytest for automated tests.
- GitHub Actions and SonarCloud for CI and static analysis.